

Activity_Run simple linear regression

May 28, 2026

1 Activity: Run simple linear regression

1.1 Introduction

As you're learning, simple linear regression is a way to model the relationship between two variables. By assessing the direction and magnitude of a relationship, data professionals are able to uncover patterns and transform large amounts of data into valuable knowledge. This enables them to make better predictions and decisions.

In this lab, you are part of an analytics team that provides insights about your company's sales and marketing practices. You have been assigned to a project that focuses on the use of influencer marketing. For this task, you will explore the relationship between your radio promotion budget and your sales.

The dataset provided includes information about marketing campaigns across TV, radio, and social media, as well as how much revenue in sales was generated from these campaigns. Based on this information, company leaders will make decisions about where to focus future marketing resources. Therefore, it is critical to provide them with a clear understanding of the relationship between types of marketing campaigns and the revenue generated as a result of this investment.

1.2 Step 1: Imports

Import relevant Python libraries and modules.

```
[2]: # Import relevant Python libraries and modules.  
  
### YOUR CODE HERE ###  
  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from statsmodels.formula.api import ols  
import statsmodels.api as sm
```

The dataset provided is a .csv file (named `marketing_sales_data.csv`), which contains information about marketing conducted in collaboration with influencers, along with corresponding sales. Assume that the numerical variables in the data are expressed in millions of dollars. As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the

.csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

Note: This is a fictional dataset that was created for educational purposes and modified for this lab.

```
[3]: # RUN THIS CELL TO IMPORT YOUR DATA.

    ### YOUR CODE HERE ###

    data = pd.read_csv("marketing_sales_data.csv")
```

Hint 1

Refer to what you learned about loading data in Python.

Hint 2

There is a function in the **pandas** library that allows you to read data from a .csv file and load the data into a DataFrame.

Hint 3

Use the `read_csv()` function from the **pandas** library.

1.3 Step 2: Data exploration

To get a sense of what the data includes, display the first 10 rows of the data.

```
[5]: # Display the first 10 rows of the data.

    ### YOUR CODE HERE ###

    data.head(10)
```

```
[5]:
```

	TV	Radio	Social Media	Influencer	Sales
0	Low	1.218354	1.270444	Micro	90.054222
1	Medium	14.949791	0.274451	Macro	222.741668
2	Low	10.377258	0.061984	Mega	102.774790
3	High	26.469274	7.070945	Micro	328.239378
4	High	36.876302	7.618605	Mega	351.807328
5	High	25.561910	5.459718	Micro	261.966812
6	High	37.263819	6.886535	Nano	349.861575
7	Low	13.187256	2.766352	Macro	140.415286
8	High	29.520170	2.333157	Nano	264.592233
9	Low	3.773287	0.135074	Nano	55.674214

Hint 1

Refer to what you learned about exploring datasets in Python.

Hint 2

There is a function in the **pandas** library that allows you to get a specific number of rows from the top of a DataFrame.

Hint 3

Use the `head()` function from the **pandas** library.

Question: What do you observe about the different variables included in the data?

I observe the following about the different variables included in the data: 1. There are 3 numeric variables and 2 categorical variables. 2. TV marketing campaigns budget is 'Low', 'Medium' or 'High'. 3. Influencer campaigns budget is 'Micro', 'Macro', 'Mega', or 'Nano'. 4. Marketing campaigns for Radio and Social Media, and Sales are in millions of dollars.

Next, to get a sense of the size of the dataset, identify the number of rows and the number of columns.

```
[6]: # Display number of rows, number of columns.
```

```
### YOUR CODE HERE ###
```

```
data.shape
```

```
[6]: (572, 5)
```

Hint 1

Refer to what you learned about exploring datasets in Python.

Hint 2

There is a property in every DataFrame in **pandas** that gives you access to the number of rows and the number of columns as a tuple.

Hint 3

Use the **shape** property.

Question: How many rows and columns exist in the data?

There are 572 rows and 5 columns in the dataset.

Now, check for missing values in the rows of the data. This is important because missing values are not that meaningful when modeling the relationship between two variables. To do so, begin by getting Booleans that indicate whether each value in the data is missing. Then, check both columns and rows for missing values.

```
[7]: # Start with .isna() to get booleans indicating whether each value in the data is missing.
```

```
### YOUR CODE HERE ###
```

```
data.isna()
```

```
[7]:
```

	TV	Radio	Social Media	Influencer	Sales
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
..	...	...	...	...	...
567	False	False	False	False	False
568	False	False	False	False	False
569	False	False	False	False	False
570	False	False	False	False	False
571	False	False	False	False	False

[572 rows x 5 columns]

If you would like to read more about the `isna()` function, refer to its documentation in the references section of this lab.

```
[8]: # Use .any(axis=1) to get booleans indicating whether there are any missing
      ↪ values along the columns in each row.

      ### YOUR CODE HERE ###

      data.isna().any(axis=1)
```

```
[8]: 0      False
      1      False
      2      False
      3      False
      4      False

      ...
      567    False
      568    False
      569    False
      570    False
      571    False
      Length: 572, dtype: bool
```

If you would like to read more about the `any()` function, refer to its documentation in the references section of this lab.

```
[9]: # Use .sum() to get the number of rows that contain missing values.

      ### YOUR CODE HERE ###

      data.isna().any(axis=1).sum()
```

```
[9]: 3
```

If you would like to read more about the `sum()` function, refer to its documentation in the references section of this lab.

Question: How many rows containing missing values?

3 rows contain missing values.

Next, drop the rows that contain missing values. Data cleaning makes your data more usable for analysis and regression. Then, check to make sure that the resulting data does not contain any rows with missing values.

```
[10]: # Use .dropna(axis=0) to indicate that you want rows which contain missing
      ↪ values to be dropped. To update the DataFrame, reassign it to the result.

      ### YOUR CODE HERE ###

      data=data.dropna(axis=0)
```

```
[11]: # Start with .isna() to get booleans indicating whether each value in the data
      ↪ is missing.
      # Use .any(axis=1) to get booleans indicating whether there are any missing
      ↪ values along the columns in each row.
      # Use .sum() to get the number of rows that contain missing values

      ### YOUR CODE HERE ###

      data.isna().any(axis=1).sum()
```

```
[11]: 0
```

The next step for this task is checking model assumptions. To explore the relationship between radio promotion budget and sales, model the relationship using linear regression. Begin by confirming whether the model assumptions for linear regression can be made in this context.

Note: Some of the assumptions can be addressed before the model is built. These will be addressed in this section. After the model is built, you will finish checking the assumptions.

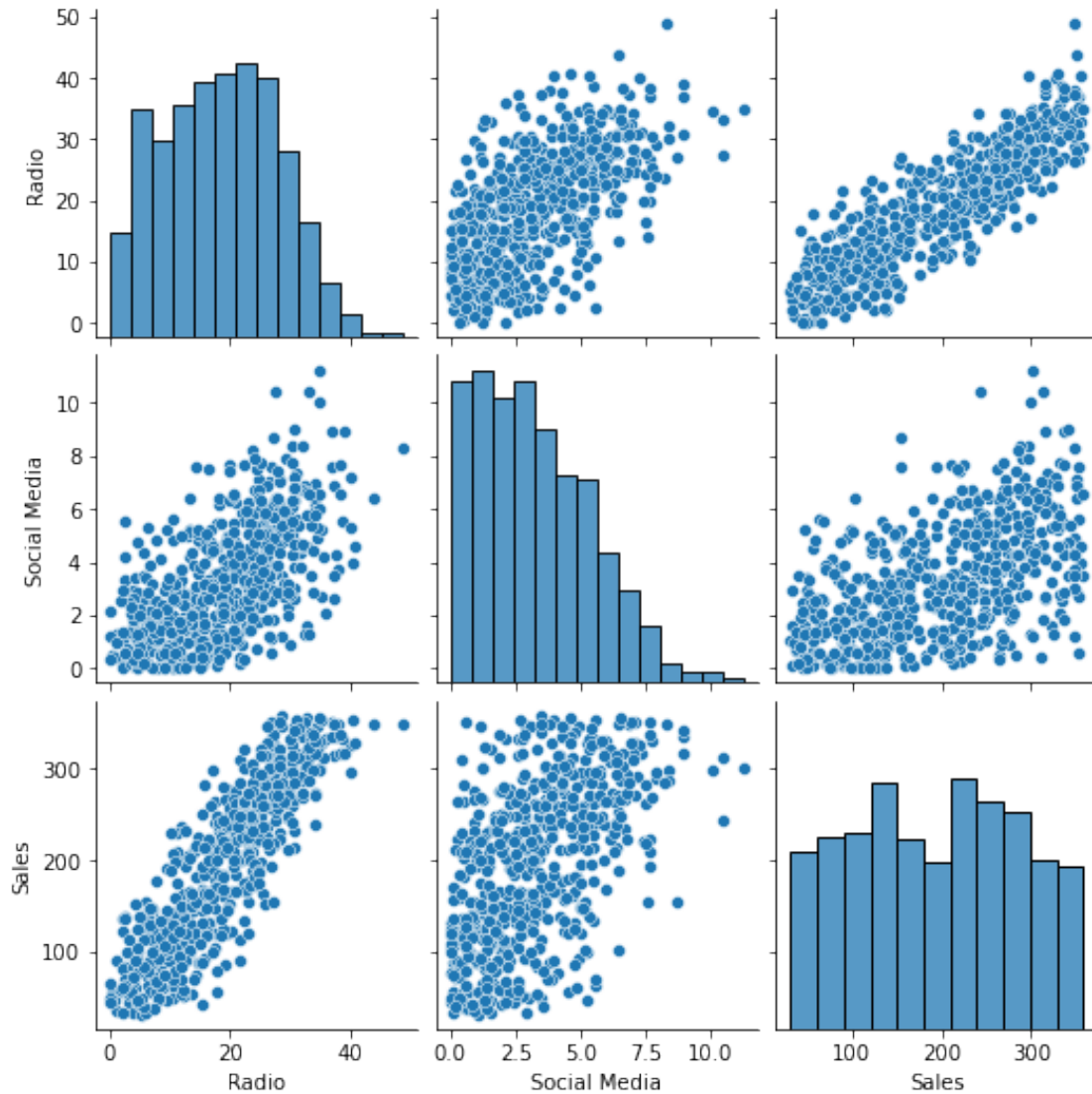
Create a plot of pairwise relationships in the data. This will help you visualize the relationships and check model assumptions.

```
[12]: # Create plot of pairwise relationships.

      ### YOUR CODE HERE ###

      sns.pairplot(data)
```

```
[12]: <seaborn.axisgrid.PairGrid at 0x769565cdf250>
```



Hint 1

Refer to the video section about creating a plot that shows the relationships between pairs of variables.

Hint 2

There is a function in the `seaborn` library that you can call to create a plot that shows the relationships between pairs of variables.

Hint 3

Call the `pairplot()` function from the `seaborn` library.

Question: Is the assumption of linearity met?

In the scatter plot of **Sales** over **Radio**, the points appear to cluster around a line that indicates a

positive association between the two variables. Since the points cluster around a line, it seems the assumption of linearity is met.

Hint 1

Refer to the video section about checking model assumptions for linear regression.

Hint 2

Use the scatterplot of **Sales** over **Radio** found in the preceding plot of pairwise relationships.

Hint 3

Check the scatterplot of **Sales** over **Radio** found in the plot of pairwise relationships. If the data points cluster around a line, that indicates that the assumption of linearity is met. Alternatively, if the data points resemble a random cloud or a curve, then a linear model may not fit the data.

1.4 Step 3: Model building

Select only the columns that are needed for the model.

```
[13]: # Select relevant columns.  
      # Save resulting DataFrame in a separate variable to prepare for regression.  
  
      ### YOUR CODE HERE ###  
  
      ols_data=data[["Radio","Sales"]]
```

Hint 1

Refer to the video about selecting multiple columns from a DataFrame.

Hint 2

Use two pairs of square brackets around the names of the columns that should be selected.

Hint 3

Make sure column names are spelled exactly as they are in the data.

Now, display the first 10 rows of the new DataFrame to better understand the data.

```
[14]: # Display first 10 rows of the new DataFrame.  
  
      ### YOUR CODE HERE ###  
  
      ols_data.head(10)
```

```
[14]:      Radio      Sales  
0    1.218354    90.054222  
1   14.949791   222.741668  
2   10.377258   102.774790  
3   26.469274   328.239378
```

```
4  36.876302  351.807328
5  25.561910  261.966812
6  37.263819  349.861575
7  13.187256  140.415286
8  29.520170  264.592233
9   3.773287   55.674214
```

Hint 1

Refer to the video about displaying contents of a DataFrame.

Hint 2

There is a function in the **pandas** library that allows you to display the first n number of rows of a DataFrame, where n is a number of your choice.

Hint 3

Call the **head()** function from the **pandas** library and pass in the number of rows from the top that you want to display.

Next, write the linear regression formula for modeling the relationship between the two variables of interest.

```
[15]: # Write the linear regression formula.
      # Save it in a variable.

      ### YOUR CODE HERE ###

      ols_formula = "Sales ~ Radio"
```

Hint 1

Refer to the video section where model building for linear regression is discussed.

Hint 2

Save the formula as string.

Hint 3

Use a tilde to separate the y variable from the x variable so that the computer understands which is which. Make sure the spelling of each variable exactly matches the corresponding column from the data.

Now, implement the ordinary least squares (OLS) approach for linear regression.

```
[16]: # Implement OLS.

      ### YOUR CODE HERE ###

      OLS = ols(formula = ols_formula, data = ols_data)
```

Hint 1

Refer to the video that discusses model building for linear regression.

Hint 2

There is a function from the `statsmodels` library that can be called to implement OLS.

Hint 3

You can call the `ols()` function from the `statsmodels` library.

Now, create a linear regression model for the data and fit the model to the data.

```
[17]: # Fit the model to the data.
      # Save the fitted model in a variable.

      ### YOUR CODE HERE ###

      model = OLS.fit()
```

Hint 1

Refer to the video section where model building for linear regression is discussed.

Hint 2

There is a function from the `statsmodels` library that can be called to fit the model.

Hint 3

Call the `fit()` function from the `statsmodels` library.

1.5 Step 4: Results and evaluation

Begin by getting a summary of the results from the model.

```
[18]: # Get summary of results.

      ### YOUR CODE HERE ###

      model.summary()
```

```
[18]: <class 'statsmodels.iolib.summary.Summary'>
      """
                                     OLS Regression Results
      =====
      Dep. Variable:                  Sales    R-squared:                  0.757
      Model:                            OLS    Adj. R-squared:              0.757
      Method:                           Least Squares    F-statistic:                1768.
      Date:                            Thu, 28 May 2026    Prob (F-statistic):         2.07e-176
      Time:                            17:03:43    Log-Likelihood:             -2966.7
      No. Observations:                  569    AIC:                        5937.
      Df Residuals:                      567    BIC:                        5946.
```

```

Df Model:                                1
Covariance Type:                        nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept    41.5326      4.067      10.211      0.000      33.544      49.521
Radio         8.1733      0.194      42.048      0.000       7.791       8.555
=====
Omnibus:                2.267   Durbin-Watson:                1.880
Prob(Omnibus):           0.322   Jarque-Bera (JB):        2.221
Skew:                   -0.102   Prob(JB):                 0.329
Kurtosis:               2.772   Cond. No.                 45.7
=====

```

Warnings:

```

[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
"""

```

Hint 1

You may find it helpful to refer back to the video section where getting model results is discussed.

Hint 2

There is a function from the `statsmodels` library that can be called to get the summary of results from a model.

Hint 3

Call the `summary()` function from the `statsmodels` library.

Next, analyze the bottom table from the results summary. Based on the table, identify the coefficients that the model determined would generate the line of best fit. The coefficients are the y-intercept and the slope.

Question: What is the y-intercept?

The y-intercept is 41.5326.

Question: What is the slope?

The slope is 8.1733.

Question: What linear equation would you write to express the relationship between sales and radio promotion budget? Use the form of $y = \text{slope} * x + \text{y-intercept}$?

The linear equation to express the relationship between sales and radio promotion budget would be $\text{Sales} = 8.1733 * \text{Radio promotion budget} + 41.5326$.

Question: What does the slope mean in this context?

It means that sales increase by 8.1733 million dollars with 1 million dollars increase in the Radio promotion budget.

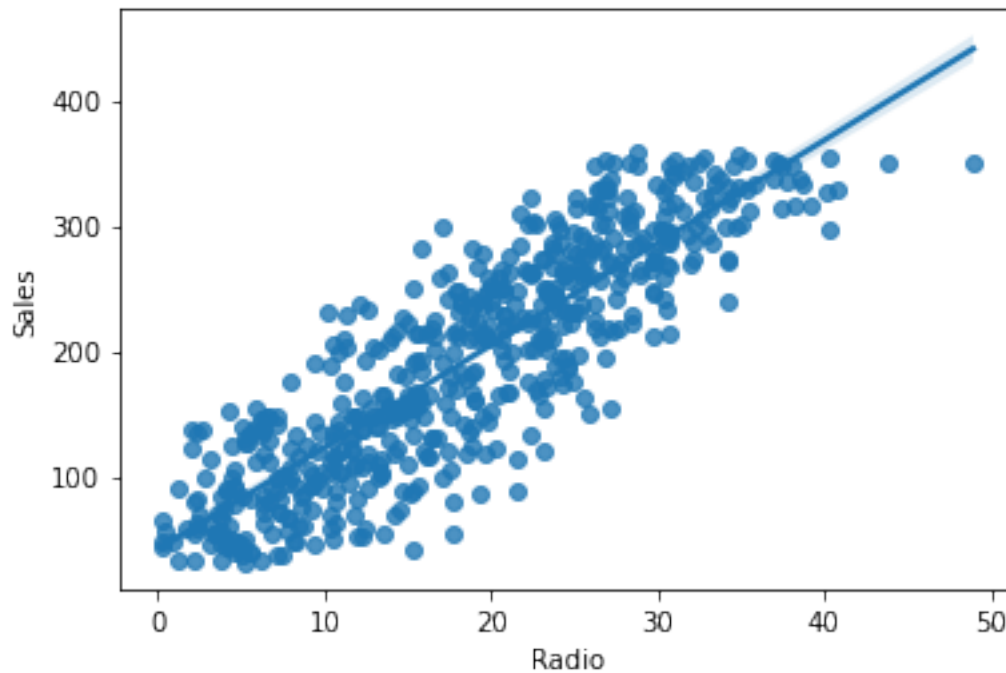
Now that you've built the linear regression model and fit it to the data, finish checking the model assumptions. This will help confirm your findings. First, plot the OLS data with the best fit regression line.

```
[19]: # Plot the OLS data with the best fit regression line.
```

```
### YOUR CODE HERE ###
```

```
sns.regplot(x = "Radio", y = "Sales", data = ols_data)
```

```
[19]: <matplotlib.axes._subplots.AxesSubplot at 0x7695637d44d0>
```



Hint 1

Refer to the video about plotting data with the best fit regression line.

Hint 2

There is a function from the `seaborn` library that can be useful here.

Hint 3

Call the `regplot()` function from the `seaborn` library.

Question: What do you observe from the preceding regression plot?

The regression plot shows that there is an approximate linear relationship between the two variables **Radio** and **Sales**. This confirms the assumption of linearity.

Now, check the normality assumption. Get the residuals from the model.

```
[20]: # Get the residuals from the model.
```

```
### YOUR CODE HERE ###
```

```
residuals = model.resid
```

Hint 1

Refer to the video about accessing residuals.

Hint 2

There is an attribute from the `statsmodels` library that can be called to get the residuals from a fitted model.

Hint 3

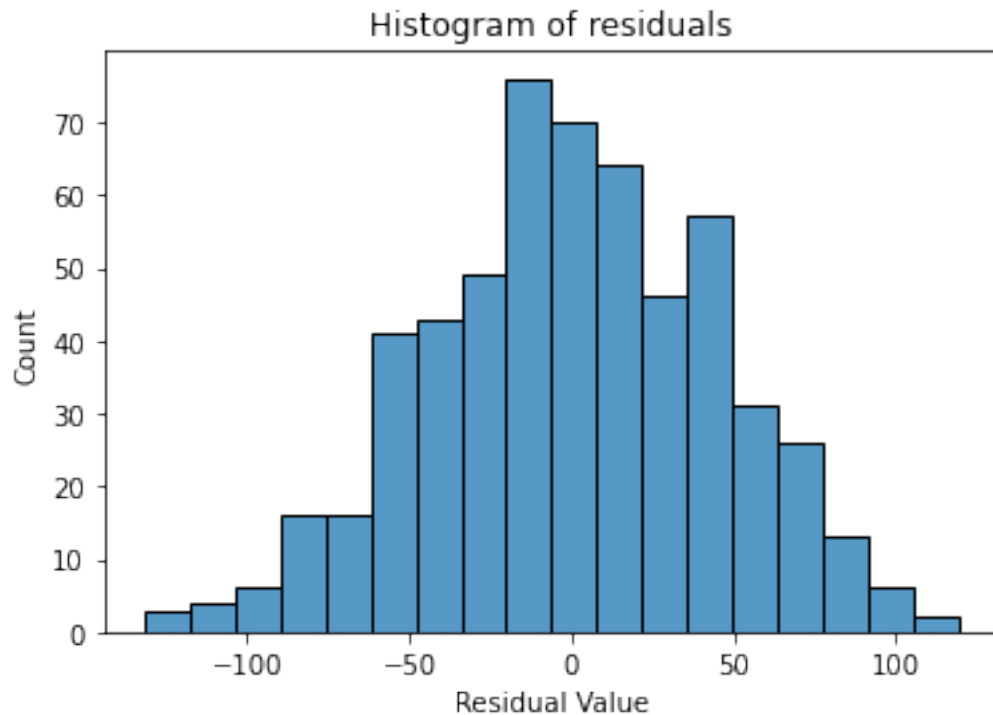
Call the `resid` attribute from the `statsmodels` library.

Now, visualize the distribution of the residuals.

```
[22]: # Visualize the distribution of the residuals.
```

```
### YOUR CODE HERE ###
```

```
fig = sns.histplot(residuals)
fig.set_xlabel("Residual Value")
fig.set_title("Histogram of residuals")
plt.show()
```



Hint 1

Refer to the video about visualizing residuals.

Hint 2

There is a function from the `seaborn` library that can be called to create a histogram.

Hint 3

Call the `histplot()` function from the `seaborn` library.

Question: Based on the visualization, what do you observe about the distribution of the residuals?

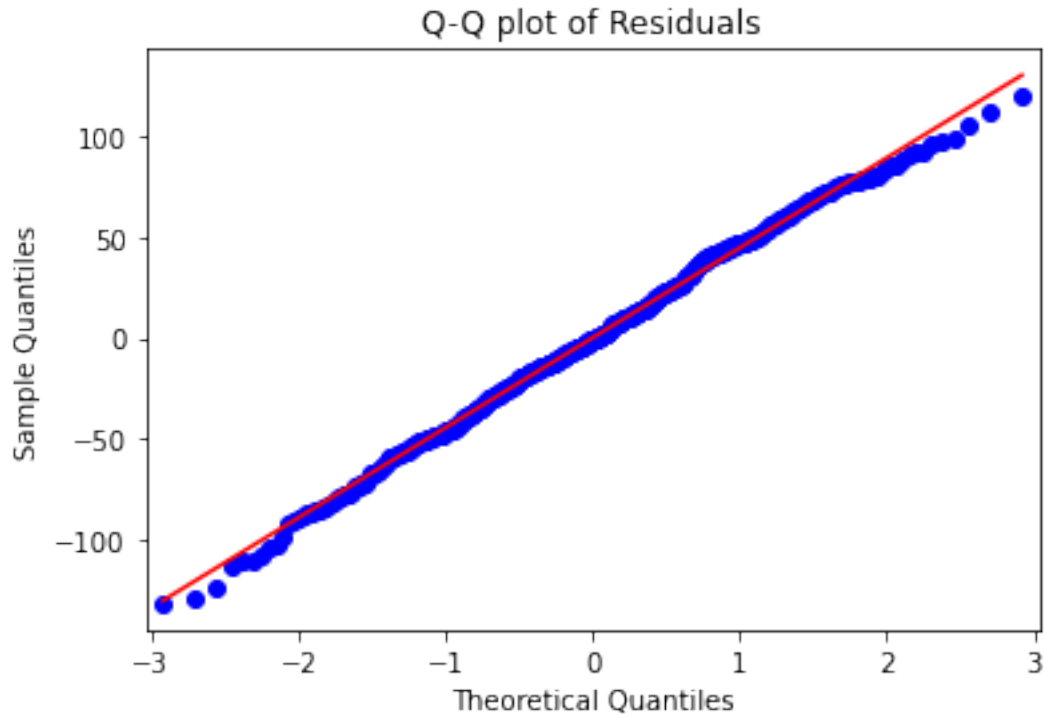
The visualization shows that the distribution of the residuals is approximately normal. This indicates that the assumption of normality is likely met.

Next, create a Q-Q plot to confirm the assumption of normality.

```
[23]: # Create a Q-Q plot.

### YOUR CODE HERE ###

sm.qqplot(residuals, line = 's')
plt.title("Q-Q plot of Residuals")
plt.show()
```



Hint 1

Refer to the video about creating a Q-Q plot.

Hint 2

There is a function from the `statsmodels` library that can be called to create a Q-Q plot.

Hint 3

Call the `qqplot()` function from the `statsmodels` library.

Question: Is the assumption of normality met?

Since the points in the Q-Q plot closely follow a straight diagonal line trending upward, the assumption of normality is met.

Now, check the assumptions of independent observation and homoscedasticity. Start by getting the fitted values from the model.

```
[25]: # Get fitted values.  
  
### YOUR CODE HERE ###  
  
fitted_values = model.predict(ols_data["Radio"])
```

Hint 1

Refer to the video about calculating fitted values.

Hint 2

There is a function from the `statsmodels` library that can be called to calculate fitted values from the model.

Hint 3

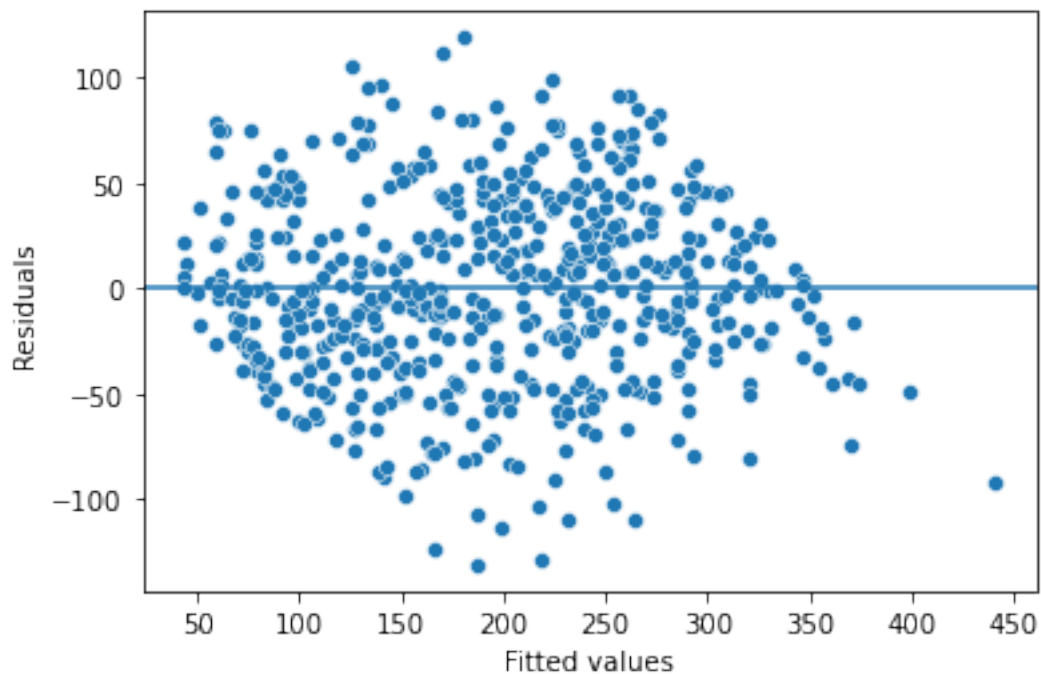
Call the `predict()` function from the `statsmodels` library. Make sure to pass in the column from `ols_data` corresponding to the x variable.

Next, create a scatterplot of the residuals against the fitted values.

```
[26]: # Create a scatterplot of residuals against fitted values.
```

```
### YOUR CODE HERE ###
```

```
fig = sns.scatterplot(x=fitted_values, y=residuals)
fig.axhline(0)
fig.set_xlabel("Fitted values")
fig.set_ylabel("Residuals")
plt.show()
```



Hint 1

Refer to the video about visualizing residuals against fitted values.

Hint 2

There is a function from the `seaborn` library that can be called to create a scatterplot.

Hint 3

Call the `scatterplot()` function from the `seaborn` library.

Question: Are the assumptions of independent observation and homoscedasticity met?

Since the data points have a cloud-like resemblance and do not follow an explicit pattern, it appears that the independent observation assumption has not been violated. And, since the residuals appear to be randomly selected, the assumption of homoscedasticity seems to be met as well.

1.6 Considerations

What are some key takeaways that you learned during this lab?

I learned how useful scatterplots and EDA can be in exploring regression models. I also learned how the results of a linear regression model can be used to express the relationship between two variables.

How would you present your findings from this lab to others?

I would share that if the company has a budget of 1 million dollars more for marketing on the radio, their sales would increase by approximately 8.1733 million dollars on average. This statistic would be significant in making marketing decisions.

What summary would you provide to stakeholders?

Based on the regression analysis of the dataset, there is a notable relationship between radio promotion budget and sales for companies in this dataset, with a p-value of 0.000 and standard error of 0.194. For these companies, a 1 million dollar increase in radio promotion budget could lead to 8.1733 million dollar increase in sales, meaning, it might be worth continuing to promote their products/services on radio.

References

[Pandas.DataFrame.Any — Pandas 1.4.3 Documentation.](#)

[Pandas.DataFrame.Isna — Pandas 1.4.3 Documentation.](#)

[Pandas.Series.Sum — Pandas 1.4.3 Documentation.](#)

[Saragih, H.S. *Dummy Marketing and Sales Data*.](#)

Congratulations! You've completed this lab. However, you may not notice a green check mark next to this item on Coursera's platform. Please continue your progress regardless of the check mark. Just click on the "save" icon at the top of this notebook to ensure your work has been logged.